

Занятие 4. Flow matching: генерация как обучение векторного поля

Какой вопрос закрывает занятие

Ось А, контекст и backbone: что хранится, как читается, как записывается, что забывается.

Ось В, генеративная конструкция: как факторизуется распределение, какой путь, какой objective, какой сэмплер.

Здесь фиксируем Ось А и меняем Ось В. На занятиях 5–6 увидим, как эти оси снова взаимодействуют.

Первые три занятия были устроены вокруг autoregressive generation. Модель строила результат token за token, а основной архитектурный вопрос звучал примерно так: **что нужно сохранить о уже обработанном prefix, чтобы получить следующий token?** Отсюда появлялись KV-cache, recurrent state, способы записи и забывания, hybrid architectures и sparse retrieval. В конце занятия 3 этот переход уже был зафиксирован явно: теперь меняется не sequence mixer, а сам тип генеративного процесса.

На занятии 4 рамка другая.

Пусть нужно сгенерировать объект $x \in \mathbb{R}^d$. Это может быть изображение, latent изображения или любой другой объект, который удобно представлять как continuous vector. Мы не будем строить его координату за координатой или token за token. Вместо этого начнём со случайного объекта x_0 из простой **source distribution** и будем постепенно менять **весь объект целиком**, пока при $\tau = 1$ не получим sample из нужной **target distribution**.

Главный объект теперь имеет вид x_τ , где $\tau \in [0, 1]$ - continuous time генеративного процесса.

Это принципиально не тот state, который встречался в первой половине курса. x_τ не является summary прошлого и не сжимает prefix. Это **сам генерируемый объект в его текущем состоянии**.

Поэтому искать здесь аналог KV-cache не нужно. Вопрос теперь такой:

можно ли обучить правило локального движения, которое переводит простое распределение в распределение данных?

Это правило будет vector field $v_\tau(x)$. Sampling сведётся к решению ODE

$$\frac{dx_\tau}{d\tau} = v_\tau(x_\tau).$$

Смена генеративного процесса меняет и то, какая координата уже введённого cost vector становится особенно заметной. На занятии 1 N_{FE} уже был выделен как число model evaluations и специально оставлен на вторую половину курса. Теперь одна генерация действительно может состоять из нескольких вызовов одной и той же сети, поэтому **NFE - Number of Function Evaluations - становится отдельной величиной, которую нельзя прятать внутри слова «шаги»**.

При этом из новой рамки не следует закон вида «памяти мало, зато проходов много». Flow-based model может иметь очень большой backbone, большой workspace и дорогой attention внутри каждого evaluation. В дальнейшем встретятся модели, где весь объект на каждом шаге представляет собой длинную последовательность. Поэтому стоимость по-прежнему сравниваем по полному cost vector, а не по метафоре «память против проходов».

Что нужно знать заранее

Предполагается, что известны:

- random variables и probability distributions;
- expectation и conditional expectation на базовом уровне;
- производная функции одной переменной;
- gradient и Jacobian;
- matrix-vector multiplication;
- MSE regression;
- neural networks и обучение через stochastic gradient descent;
- PyTorch.

Полезно помнить material занятий 1-3, прежде всего distinction между математическим объектом и алгоритмом его вычисления и cost vector.

Знание diffusion, score matching, stochastic differential equations - **SDE** - или diffusion noise schedules не предполагается. Они появятся на следующем занятии.

ODE тоже не предполагается как отдельная пройденная тема. Всё, что понадобится про ordinary differential equations, вводится здесь.

Что должно получаться после чтения

После этого конспекта нужно уметь без дополнительных материалов:

1. Объяснить разницу между autoregressive state S_t из первой половины купса и generative state x_τ .
 2. По source distribution p_0 , target distribution p_1 и заданному family p_τ объяснить, что такое probability path.
 3. Объяснить, что задаёт coupling и почему одинаковые p_0 и p_1 ещё не определяют единственный способ соединить samples.
 4. Перейти от vector field $v_\tau(x)$ к ODE и записать trajectory в integral form.
 5. Для linear interpolation $x_\tau = (1 - \tau)x_0 + \tau x_1$ самостоятельно получить conditional velocity $x_1 - x_0$.
 6. Для interpolation $x_\tau = a_\tau x_0 + b_\tau x_1$ получить target $\dot{a}_\tau x_0 + \dot{b}_\tau x_1$.
 7. Записать Conditional Flow Matching loss и объяснить, почему для построения одного training target не нужно решать ODE.
 8. Полностью вывести, почему MSE-regression возвращает conditional expectation.
 9. Объяснить, почему $v^*(x, \tau) = \mathbb{E}[U_\tau \mid X_\tau = x]$ является именно тем marginal field, который нужен при sampling.
 10. Различить straight training interpolation и trajectory обученного marginal ODE.
 11. Объяснить, почему straight conditional interpolations не гарантируют straight learned trajectories и что дополнительная процедура reflow меняет в coupling и trajectory geometry.
 12. Вывести Euler update из определения derivative.
 13. Различить solver step и NFE и посчитать оба числа для простых numerical solvers.
 14. Объяснить, почему small-NFE generation зависит и от learned field, и от numerical integrator.
 15. Объяснить, почему ни flow matching, ни diffusion как классы сами по себе не задают универсального memory profile.
-

Словарь занятия

Понятие	English term	Определение
Исходное распределение	source distribution	Простое распределение p_0 , из которого удобно независимо получать samples перед началом генерации. Частый пример - standard Gaussian.
Целевое распределение	target distribution	Распределение p_1 , samples из которого хотим генерировать. В generative modeling обычно соответствует data distribution или её approximation.
Путь распределений	probability path	Семейство distributions $\{p_\tau\}_{\tau \in [0,1]}$, которое соединяет p_0 и p_1 .
Связка	coupling	Joint distribution пары (X_0, X_1) с заданными marginals p_0 и p_1 . Coupling говорит, какие source samples считаются связанными с какими target samples.
Векторное поле	vector field	Функция $v_\tau(x)$, которая каждой точке x и времени τ сопоставляет локальную velocity.
Обыкновенное дифференциальное уравнение	ordinary differential equation, ODE	Уравнение, задающее derivative состояния через его текущее значение, например $\frac{dx_\tau}{d\tau} = v_\tau(x_\tau)$.
Поток	flow	Семейство отображений Φ_τ , получаемых интегрированием ODE: $\Phi_\tau(x_0)$ показывает, куда к времени τ придёт initial point x_0 .
Непрерывный normalizing flow	Continuous Normalizing Flow, CNF	Generative model, в которой learnable vector field задаёт continuous invertible transport через ODE.
Условный путь	conditional probability path	Более простой probability path, заданный при фиксированной conditioning information, например при известных endpoints.
Интерполяция	interpolation	Явная формула для промежуточного объекта между endpoints, например $(1 - \tau)x_0 + \tau x_1$.
Условное векторное поле	conditional vector field	Velocity field конкретного conditional path. При явной interpolation его target часто получается аналитически.
Conditional Flow Matching	Conditional Flow Matching, CFM	Training objective, в котором neural vector field перепецирует на analytically available conditional velocities.

Понятие	English term	Определение
Маргинальное поле	marginal vector field	Поле, зависящее только от текущих x, τ и генерирующее marginal probability path после усреднения скрытой conditioning information.
Регрессионная проекция	regression projection	Свойство squared-error regression: лучший predictor скрытой target variable по наблюдаемому input равен conditional expectation target при этом input.
Rectified Flow	Rectified Flow	Подход с regression на velocities straight conditional interpolations; reflow является дополнительной процедурой rectification coupling, а не обязательным условием первого fit.
Прямая интерполяция	straight interpolation	Conditional path $X_\tau = (1 - \tau)X_0 + \tau X_1$ с постоянной conditional velocity.
Траектория	trajectory	Конкретная curve $\tau \mapsto x_\tau$, полученная интегрированием vector field из некоторого initial point.
Reflow	reflow / rectification	Повторное обучение Rectified Flow на новом coupling, индуцированном предыдущим learned flow.
Кривизна траектории	trajectory curvature	Насколько velocity вдоль learned trajectory меняет направление. В этом занятии используется как интуитивная характеристика departure от straight constant-speed motion.
Численный ODE solver	numerical ODE solver	Алгоритм, приближённо интегрирующий ODE по конечному числу evaluations vector field.
Шаг Euler	Euler step	First-order update $x^{(m+1)} = x^{(m)} + \Delta\tau v_{\tau_m}(x^{(m)})$.
Шаг solver	solver step	Один переход numerical solver от текущего grid point к следующему. Внутри такого шага может быть несколько field evaluations.
Число вычислений функции	Number of Function Evaluations, NFE	Число вызовов learned vector field при генерации sample.

1. Probability path и vector field

Зачем эта тема здесь

До сих пор генерация была дискретной по position index t . Сначала существовал prefix длины $t - 1$, затем модель вычисляла следующий token, после чего prefix увеличивался.

Если теперь сказать только «мы будем постепенно превращать noise в data», остаются неясными три вещи.

Во-первых, что именно значит «постепенно» на уровне distributions?

Во-вторых, что движет отдельный sample?

В-третьих, как движение отдельного sample связано с тем, что **вся population samples** должна в конце иметь правильное distribution?

Для этого нужны два разных объекта: probability path и vector field.

Интуиция

Представим облако точек в $\mathbb{R}^2$.

В момент $\tau = 0$ это, например, почти круглая Gaussian cloud. В момент $\tau = 1$ хочется получить cloud в форме данных - допустим, две вытянутые группы.

Probability path отвечает на вопрос:

как должно выглядеть всё облако в каждый промежуточный момент?

Vector field отвечает на другой вопрос:

если конкретная частица сейчас находится в точке x , куда и с какой скоростью ей двигаться?

Если vector field выбран правильно, движение всех частиц одновременно меняет их distribution вдоль нужного probability path.

То есть есть два уровня:

distribution dynamics $p_0 \rightarrow p_\tau \rightarrow p_1$

и

sample dynamics $x_0 \rightarrow x_\tau \rightarrow x_1$.

Их нельзя отождествлять. p_τ - distribution. x_τ - одна realization.

Понятия, которые вводятся

Нужны source distribution p_0 , target distribution p_1 , probability path p_τ , coupling, vector field v_τ , trajectory, flow map, ODE и CNF.

Формальное изложение

Source и target distributions

Пусть data space равен $\mathbb{R}^d$.

Выбираем простое source distribution $X_0 \sim p_0$.

Например, $p_0 = \mathcal{N}(0, I)$.

Target distribution обозначаем $X_1 \sim p_1$.

В идеальном generative problem p_1 - data distribution. На практике мы имеем finite dataset и samples из неизвестного underlying distribution, а не аналитическую формулу его density.

Нам нужно построить random transformation, после которой

$$X_0 \sim p_0 \implies X_1 \sim p_1.$$

Probability path

Вместо одного резкого transformation рассматриваем family p_τ , $\tau \in [0, 1]$, с boundary conditions $p_{\tau=0} = p_0$, $p_{\tau=1} = p_1$.

Такое family называется **probability path**.

Важно: endpoints не определяют path однозначно.

Между двумя distributions можно построить множество разных continuous paths. Один может сильно раздувать distribution посередине, другой - почти напрямую транспортировать mass, третий - вращать её или делать ещё что-то лишнее.

Поэтому выбор probability path - содержательная часть generative method.

Coupling

Ещё раньше появляется другой выбор.

Пусть у нас есть random pair $(X_0, X_1) \sim \pi$.

Distribution π называется coupling distributions p_0 и p_1 , если её marginals равны нужным endpoints: $X_0 \sim p_0$, $X_1 \sim p_1$.

Самый простой coupling - independent: $\pi = p_0 \otimes p_1$.

То есть noise sample и data sample выбираются независимо.

Но это только один вариант. Можно связывать source и target более содержательно.

Почему coupling важен, станет особенно видно в Rectified Flow. Если мы рисуем segment от каждого x_0 к связанному с ним x_1 , то именно coupling определяет, какие segments вообще появятся.

Vector field

Пусть задана функция $v_\tau : \mathbb{R}^d \rightarrow \mathbb{R}^d$.

Для каждой точки x она возвращает vector той же dimensionality. Интерпретация:

$$v_\tau(x) = \text{velocity частицы, находящейся в } x \text{ в момент } \tau.$$

Trajectory определяется ODE

$$\frac{dx_\tau}{d\tau} = v_\tau(x_\tau).$$

В левой части стоит фактическая instantaneous velocity trajectory. Правая часть говорит, какую velocity предписывает field в текущей точке.

От derivative к trajectory

Из fundamental theorem of calculus получаем integral form ODE:

$$x_\tau = x_0 + \int_0^\tau v_s(x_s) ds.$$

Этот переход стоит прочитать буквально.

Чтобы узнать x_τ , недостаточно посмотреть на v_τ один раз. Нужно накопить все infinitesimal displacements вдоль уже пройденной trajectory.

Если field constant, $v_\tau(x) = c$, то

$$x_\tau = x_0 + \int_0^\tau c ds = x_0 + \tau c.$$

Получилась straight line с constant velocity.

Если field меняется с x или τ , trajectory в общем случае curved.

Flow map

При стандартных regularity assumptions ODE с initial condition x_0 имеет unique solution.

Тогда можно определить map $\Phi_\tau(x_0) = x_\tau$.

Это **flow map**.

Если initial random variable имеет distribution p_0 , то $X_\tau = \Phi_\tau(X_0)$ тоже является random variable. Его distribution обозначаем p_τ .

Говорят, что flow переносит - **pushes forward** - p_0 в p_τ : $p_\tau = (\Phi_\tau)_\# p_0$.

Если в конце $(\Phi_1)_\# p_0 = p_1$, то sampling устроен просто:

1. получить $x_0 \sim p_0$;
2. решить ODE от $\tau = 0$ до $\tau = 1$;
3. вернуть $x_1 = \Phi_1(x_0)$.

Что здесь берём как факт

Для differentiable vector field $v : \mathbb{R}^d \rightarrow \mathbb{R}^d$ divergence определяется как trace Jacobian:

$$\nabla_x \cdot v(x) = \text{tr } J_v(x) = \sum_j \frac{\partial v_j(x)}{\partial x_j}.$$

Чтобы связать field с evolution density строго, используется **continuity equation**

$$\frac{\partial p_\tau(x)}{\partial \tau} + \nabla_x \cdot (p_\tau(x) v_\tau(x)) = 0.$$

Интуитивно это conservation of probability mass: probability не возникает и не исчезает, а только течёт через space под действием field.

Полное доказательство continuity equation здесь не нужно.

Для Continuous Normalizing Flow также используется instantaneous change-of-variables formula. Вдоль trajectory она имеет вид

$$\frac{d}{d\tau} \log p_\tau(x_\tau) = -\nabla_x \cdot v_\tau(x_\tau).$$

Отсюда

$$\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \nabla_x \cdot v_\tau(x_\tau) d\tau.$$

Эту формулу тоже берём как факт. Она нужна, чтобы понимать, почему ODE-based transport связан с normalizing flows и density evaluation, но вывод determinant identities в этом занятии ничего не добавит к основной идее.

Continuous Normalizing Flows в современном виде были сформулированы в Neural ODE framework; Flow Matching затем дал simulation-free training objective для таких vector fields.

Пример: scaling Gaussian через простое поле

Пусть $X_0 \sim \mathcal{N}(0, 1)$ и выберем one-dimensional field $v_\tau(x) = ax$, где a - constant.

ODE:

$$\frac{dx_\tau}{d\tau} = ax_\tau.$$

Его solution можно проверить напрямую: $x_\tau = e^{a\tau} x_0$.

Действительно,

$$\frac{d}{d\tau} e^{a\tau} x_0 = a e^{a\tau} x_0 = ax_\tau.$$

Поскольку Gaussian random variable просто умножается на scalar,

$$X_\tau \sim \mathcal{N}(0, e^{2a\tau}).$$

Хотим в момент $\tau = 1$ получить variance 4. Тогда нужно $e^{2a} = 4$.

Следовательно, $a = \log 2$.

Получается

$$X_\tau \sim \mathcal{N}(0, 2^{2\tau})$$

и $X_1 \sim \mathcal{N}(0, 4)$.

Этот маленький пример показывает всю конструкцию: field задаёт trajectories отдельных points, а ensemble trajectories задаёт probability path.

Где здесь легко ошибиться

« x_τ - это новый recurrent state вместо S_t »

Нет.

S_t в первой половине курса был representation уже увиденного prefix. Его задача - сохранить информацию для следующего autoregressive step.

x_τ здесь - сам sample, который мы генерируем.

Увеличение τ не означает, что модель увидела ещё один token. Оно означает, что текущий объект продвинулся дальше вдоль generative dynamics.

« τ - просто новое обозначение позиции t »

Нет.

t остаётся token position.

τ - continuous generative time.

В одной и той же future model могут одновременно существовать sequence positions t внутри backbone и diffusion/flow time τ . Смешивать их особенно неприятно, потому что оба индекса могут входить в neural network одновременно.

«Probability path и trajectory - одно и то же»

Нет.

Trajectory - одна curve $\tau \mapsto x_\tau$.

Probability path - distribution всех X_τ в каждый момент.

У одной probability path может быть больше одного generating vector field.

«Если выбраны p_0 и p_1 , всё остальное определено»

Нет.

Нужно выбрать хотя бы path между ними. Если construction использует paired endpoints, имеет значение ещё и coupling.

«Раз model итеративная, у неё автоматически мало памяти»

Нет.

ODE говорит, как связан current sample между evaluations. Оно ничего не говорит о memory footprint neural network, которая вычисляет $v_\theta(x, \tau)$.

Можно построить field с дешёвым маленьким MLP, а можно - с огромным Transformer backbone. Класс generative dynamics не фиксирует эту координату cost vector.

Что теперь понятно

Generative model можно определить не как conditional distribution следующего token, а как continuous transport.

Нам нужен vector field, который двигает individual samples так, чтобы их marginals прошли от p_0 к p_1 .

Но здесь возникает практическая проблема: откуда взять правильный field для supervision?

Если для каждого training example пришлось бы сначала решать другой ODE, training выглядел бы довольно странно. Следующий раздел как раз убирает эту проблему.

2. Conditional path и Conditional Flow Matching

Зачем эта тема здесь

Мы сформулировали идеальную задачу:

выучить $v_\theta(x, \tau)$, генерирующий нужный p_τ .

Но target field для marginal distribution обычно неизвестен.

Data даёт samples x_1 . Source distribution даёт samples x_0 . Оно не выдаёт готовую стрелку $v_\tau(x)$ для произвольной intermediate point.

Flow Matching делает полезный обходной ход.

Вместо того чтобы сразу строить сложное marginal field, построим **простые conditional paths**, velocities которых известны аналитически.

Интуиция

Берём pair endpoints: $x_0 \sim p_0$, $x_1 \sim p_1$.

Теперь временно представим, что оба endpoint нам известны.

Тогда самый простой способ соединить их - провести segment.

В момент $\tau = 0$ мы находимся в x_0 .

В момент $\tau = 1$ - в x_1 .

Между ними движемся равномерно.

У этого движения velocity не нужно угадывать: её можно получить обычной производной.

Получается supervised learning problem:

- input - случайная intermediate point x_τ и время τ ;
- target - известная velocity conditional path;
- predictor - neural network $v_\theta(x_\tau, \tau)$.

Главное удобство здесь такое: **training point и target строятся closed-form, без numerical ODE solve.**

ODE понадобится позже, при sampling.

Понятия, которые вводятся

Нужны conditional probability path, interpolation, conditional vector field и Conditional Flow Matching.

Формальное изложение

Straight interpolation

Пусть coupling задаёт random pair $(X_0, X_1) \sim \pi$.

Для конкретной realization (x_0, x_1) определим $x_\tau = (1 - \tau)x_0 + \tau x_1$.

Проверим endpoints.

При $\tau = 0$: $x_{\tau=0} = x_0$.

При $\tau = 1$: $x_{\tau=1} = x_1$.

Теперь найдём velocity.

Распишем interpolation: $x_\tau = x_0 - \tau x_0 + \tau x_1$.

Дифференцируем по τ :

$$\begin{aligned}\frac{dx_\tau}{d\tau} &= \frac{d}{d\tau}x_0 - \frac{d}{d\tau}(\tau x_0) + \frac{d}{d\tau}(\tau x_1) \\ &= 0 - x_0 + x_1.\end{aligned}$$

Следовательно,

$$u_\tau = x_1 - x_0.$$

Velocity не зависит от τ .

Это и есть причина, почему interpolation straight не только геометрически, но и в смысле constant-speed motion.

Почему это conditional field

Target $x_1 - x_0$ знает оба endpoints.

Во время training это нормально: мы сами sampled pair и поэтому знаем её полностью.

Во время generation target endpoint x_1 неизвестен - его как раз нужно получить.

Поэтому u_τ здесь является **conditional velocity**. Она доступна при условии знания конкретной training pair, но model при sampling такой информации иметь не будет.

Это distinction станет центральным в следующем разделе.

CFM loss

Training procedure можно записать так: $(X_0, X_1) \sim \pi$, $\tau \sim \mathcal{U}[0, 1]$, $X_\tau = (1 - \tau)X_0 + \tau X_1$, $U_\tau = X_1 - X_0$.

Model получает только X_τ и τ : $v_\theta(X_\tau, \tau)$.

Squared-error objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E} \left[\|v_\theta(X_\tau, \tau) - (X_1 - X_0)\|_2^2 \right].$$

Expectation идёт по sampled time и endpoint pairs.

Почему training называется simulation-free

Чтобы получить один training sample, нужно выполнить только algebraic interpolation: $X_\tau = (1 - \tau)X_0 + \tau X_1$.

Target тоже algebraic: $U_\tau = X_1 - X_0$.

Нам не нужно начинать с X_0 , численно делать 10, 50 или 100 ODE steps до τ и только затем вычислять loss.

То есть **не требуется ODE simulation для construction training target**.

Это не значит, что ODE исчезает из model. Sampling после обучения всё ещё должен интегрировать learned field.

Flow Matching paper именно в этом смысле называет метод simulation-free training of CNFs.

Общая interpolation

Теперь пусть $x_\tau = a_\tau x_0 + b_\tau x_1$, где a_τ и b_τ - differentiable scalar functions.

Чтобы endpoints совпадали с x_0 и x_1 , естественно потребовать $a_0 = 1$, $b_0 = 0$, $a_1 = 0$, $b_1 = 1$.

Дифференцируем:

$$\begin{aligned} \frac{dx_\tau}{d\tau} &= \frac{d}{d\tau} (a_\tau x_0 + b_\tau x_1) \\ &= \dot{a}_\tau x_0 + \dot{b}_\tau x_1. \end{aligned}$$

Поэтому conditional velocity равна

$$u_\tau = \dot{a}_\tau x_0 + \dot{b}_\tau x_1.$$

Для straight interpolation $a_\tau = 1 - \tau$, $b_\tau = \tau$.

Тогда $\dot{a}_\tau = -1$, $\dot{b}_\tau = 1$ и снова $u_\tau = x_1 - x_0$.

То есть CFM target получается обычным differentiation выбранной interpolation.

Пример

Возьмём $x_0 = (-1, 0)$, $x_1 = (3, 2)$.

Straight interpolation: $x_\tau = (1 - \tau)(-1, 0) + \tau(3, 2)$.

При $\tau = \frac{1}{4}$:

$$\begin{aligned}x_{1/4} &= \frac{3}{4}(-1, 0) + \frac{1}{4}(3, 2) \\ &= (0, \frac{1}{2}).\end{aligned}$$

Conditional velocity: $u_\tau = x_1 - x_0 = (4, 2)$.

Она одинакова в любой точке этого training segment.

Training example имеет вид

$$\left((0, \frac{1}{2}), \frac{1}{4} \right) \mapsto (4, 2).$$

Для model это обычная vector regression problem.

Где здесь легко ошибиться

«Conditional» здесь означает text conditioning

Не обязательно.

Слово conditional в CFM относится прежде всего к conditioning information, которая определяет простой path и его velocity.

Text prompt, class label и другие semantic conditions можно добавить отдельно. Это другая axis.

«При training model знает x_1 , значит при sampling ей тоже нужен x_1 »

Нет.

Target construction использует endpoints, а predictor получает только текущий state и time: $v_\theta(X_\tau, \tau)$.

Именно поэтому нужен следующий раздел: модель должна преобразовать обучающий сигнал, зависящий от скрытой пары, в поле без доступа к этой паре.

«Flow Matching training вообще не связан с ODE»

Связан.

Мы обучаем right-hand side ODE.

Не требуется только интегрировать ODE для construction каждого target.

«Straight interpolation уже и есть learned trajectory»

Пока нет.

Мы определили random training curve X_τ . Learned trajectory появится только после того, как возьмём fitted v_θ и начнём решать другое equation:

$$\frac{dZ_\tau}{d\tau} = v_\theta(Z_\tau, \tau).$$

X_τ и Z_τ нужно держать разными symbols именно потому, что это разные объекты.

Что теперь понятно

Conditional target получить легко.

Можно sample endpoints, sample time, аналитически построить intermediate point и аналитически вычислить velocity.

Осталась самая важная conceptual проблема занятия.

Model получает одну и ту же форму input (x, τ) независимо от того, **какая hidden endpoint pair** породила эту точку. Если через одну точку проходят несколько training paths с разными velocities, какой target должна предсказывать model?

Ответ даёт conditional expectation.

3. Почему conditional regression даёт marginal field

Зачем эта тема здесь

До этого места легко получить неправильную картину.

Кажется, что model просто запоминает множество arrows $x_1 - x_0$, нарисованных вдоль straight segments, а затем повторяет их при sampling.

Но во время sampling никакого x_1 ещё нет.

Если две разные endpoint pairs в один и тот же момент τ дают одинаковую intermediate point x , model получает одинаковый input (x, τ) , но training targets могут отличаться.

Одна deterministic network не может одновременно вернуть два разных vectors.

Нужно понять, что squared-error regression делает в такой ситуации.

Интуиция

Представим перекрёсток.

Через него проходят несколько training segments.

Один segment хочет двигаться вправо-вверх.

Другой - вправо-вниз.

Но learned vector field обязан присвоить перекрёстку **одну** arrow для данного τ .

MSE выбирает average target среди всех hidden training situations, которые могли привести к этому observed input.

Это и есть conditional expectation.

Получается важный переход:

$$\text{pair-specific velocity} \longrightarrow \text{average local velocity given current state.}$$

Первая зависит от endpoints.

Вторая зависит только от x и τ .

Именно вторая годится для autonomous sampling procedure.

Понятия, которые вводятся

Нужны marginal vector field, conditional expectation и regression projection.

Формальное изложение

Общая MSE regression problem

Пусть Z - observed input, а U - random target vector.

Хотим выбрать function f для minimization

$$\mathcal{L}(f) = \mathbb{E} [\|f(Z) - U\|_2^2].$$

В нашем случае $Z = (X_\tau, \tau)$ и $U = U_\tau$.

Определим $m(Z) = \mathbb{E}[U \mid Z]$.

Покажем, что optimal squared-error predictor равен m .

Шаг 1. Добавим и вычтем conditional mean

Имеем $U - f(Z) = U - m(Z) + m(Z) - f(Z)$.

Поэтому

$$\begin{aligned} \|U - f(Z)\|_2^2 &= \|U - m(Z)\|_2^2 \\ &+ \|m(Z) - f(Z)\|_2^2 + 2(U - m(Z))^\top (m(Z) - f(Z)). \end{aligned}$$

Шаг 2. Возьмём expectation при фиксированном Z

Условимся на конкретном observed Z .

Тогда $m(Z) - f(Z)$ уже deterministic vector относительно conditional randomness.

Поэтому cross term превращается в

$$\begin{aligned} & \mathbb{E} [(U - m(Z))^{\top} (m(Z) - f(Z)) \mid Z] \\ &= (m(Z) - f(Z))^{\top} \mathbb{E}[U - m(Z) \mid Z]. \end{aligned}$$

Но по definition $m(Z) = \mathbb{E}[U \mid Z]$.

Следовательно, $\mathbb{E}[U - m(Z) \mid Z] = 0$.

Cross term исчезает.

Получаем

$$\begin{aligned} & \mathbb{E} [\|U - f(Z)\|_2^2 \mid Z] \\ &= \mathbb{E} [\|U - m(Z)\|_2^2 \mid Z] + \|m(Z) - f(Z)\|_2^2. \end{aligned}$$

Шаг 3. Минимизируем

Первое слагаемое не зависит от f .

Второе слагаемое неотрицательно и обращается в ноль только когда $f(Z) = m(Z)$.

Значит,

$$f^*(Z) = \mathbb{E}[U \mid Z].$$

Для flow training:

$$v^*(x, \tau) = \mathbb{E}[U_{\tau} \mid X_{\tau} = x, \tau].$$

Если τ в рассуждении считается фиксированным, запись обычно сокращают:

$$v^*(x, \tau) = \mathbb{E}[U_{\tau} \mid X_{\tau} = x].$$

Для straight interpolation: $U_{\tau} = X_1 - X_0$.

Поэтому

$$v^*(x, \tau) = \mathbb{E}[X_1 - X_0 \mid X_{\tau} = x, \tau].$$

Что означает эта формула

Model не восстанавливает hidden pair.

Она не отвечает на вопрос:

из какого именно x_0 пришла эта point и в какой именно x_1 она должна попасть?

Она отвечает на другой вопрос:

если известно, что в момент τ training interpolation находится в x , какова средняя velocity среди всех compatible endpoint pairs?

Это уже function только от текущего state и time.

Conditional loss и marginal loss

Из предыдущего decomposition следует ещё один полезный факт.

Пусть

$$v^*(X_\tau, \tau) = \mathbb{E}[U_\tau \mid X_\tau, \tau].$$

Тогда

$$\begin{aligned} & \mathbb{E} [\|v_\theta(X_\tau, \tau) - U_\tau\|_2^2] \\ &= \mathbb{E} [\|v_\theta(X_\tau, \tau) - v^*(X_\tau, \tau)\|_2^2] \\ & \quad + \mathbb{E} [\|U_\tau - v^*(X_\tau, \tau)\|_2^2]. \end{aligned}$$

Последнее слагаемое не зависит от θ .

Поэтому regression на conditional velocities и regression непосредственно на marginal field имеют одинаковый optimum, а при соответствующих assumptions - одинаковый gradient по model parameters.

Но этого всё ещё недостаточно для generative modeling.

Мы доказали regression statement. Мы ещё не доказали transport statement.

Где нужна Flow Matching theorem

В общем виде основная формула не требует существования ordinary conditional densities. Пусть U_τ - random conditional velocity, определённая совместной construction training path. Тогда marginal field задаётся conditional expectation:

$$v_\tau(x) = \mathbb{E}[U_\tau \mid X_\tau = x].$$

Density-формулы ниже являются дополнительной записью **только если conditional paths имеют densities** относительно общей reference measure. Если condition содержит всю пару endpoints и путь при фиксированных endpoints детерминирован, conditional law X_τ может быть singular, в частности Dirac measure; тогда отношение ordinary densities не является основной формой аргумента.

Пусть conditioning variable обозначена Ξ .

Для каждого ξ есть conditional path $p_\tau(x \mid \xi)$ и generating conditional field $u_\tau(x \mid \xi)$.

Marginal path получается mixing:

$$p_\tau(x) = \int p_\tau(x \mid c) q(c) dc.$$

Естественный posterior weight конкретного condition ξ при наблюдаемой точке x равен

$$\frac{p_\tau(x \mid c) q(c)}{p_\tau(x)}.$$

Поэтому marginal field имеет вид

$$v_\tau(x) = \int u_\tau(x | c) \frac{p_\tau(x | c)q(c)}{p_\tau(x)} dc.$$

Это та же conditional expectation.

При density assumption posterior-weighted формула выше является конкретной записью того же marginal field. В общем случае Flow Matching marginalization result формулируется через conditional expectation: при соответствующих regularity conditions marginal field $v_\tau(x) = \mathbb{E}[U_\tau | X_\tau = x]$ генерирует нужный marginal probability path.

То есть $v_\tau(x) = \mathbb{E}[U_\tau | X_\tau = x]$ не просто лучший MSE predictor. Его ODE действительно переносит mass вдоль нужных marginals p_τ . Это центральный мост между supervised regression и generative dynamics. В исходной Flow Matching работе это оформлено как marginalization result для conditional vector fields; там же CFM objective связывается с marginal FM objective.

Полное доказательство через уравнение непрерывности здесь не воспроизводим.

Почему без theorem вся конструкция не работала бы

MSE theorem само по себе является statement только о prediction error.

Оно говорит:

лучший predictor hidden velocity = conditional mean velocity.

Но generative model требует намного более сильное:

ODE этого predictor сохраняет нужные marginals.

Именно второе утверждение даёт Flow Matching theorem.

Без него у нас был бы аккуратный regression objective, но не было бы основания считать его generative transport method.

Геометрический пример: два crossing segments

Возьмём две equally probable endpoint pairs.

Первая: $x_0^{(A)} = (-1, -1)$, $x_1^{(A)} = (1, 1)$.

Её velocity: $u^{(A)} = (2, 2)$.

Вторая: $x_0^{(B)} = (-1, 1)$, $x_1^{(B)} = (1, -1)$.

Её velocity: $u^{(B)} = (2, -2)$.

Обе straight interpolations при

$$\tau = \frac{1}{2}$$

проходят через $x = (0, 0)$.

Значит model получает один и тот же input

$$\left((0, 0), \frac{1}{2} \right),$$

но два possible targets: $(2, 2)$ и $(2, -2)$.

При равных probabilities MSE optimum:

$$\begin{aligned} v^*((0, 0), \frac{1}{2}) &= \frac{1}{2}(2, 2) + \frac{1}{2}(2, -2) \\ &= (2, 0). \end{aligned}$$

Полученная arrow не совпадает ни с одним из двух conditional targets.

Это не ошибка regression.

Это **ровно marginalization**, которая нужна, чтобы получить single-valued vector field.

Пример дискретный и специально геометрический, поэтому не нужно использовать его как proof regularity theorem. Его задача - показать, почему learned marginal field не обязано локально повторять каждую training segment.

Где здесь легко ошибиться

«Model должна угадать правильный endpoint»

Нет.

Endpoint вообще не обязан быть identifiable из текущей point.

Model учит local marginal velocity.

«Average velocities разрушает target distribution»

Не следует.

Как раз substantive theorem Flow Matching говорит, что правильно взвешенное marginal field сохраняет нужный marginal probability path.

«Раз CFM target известен точно, fitted network должна возвращать его точно»

Только если conditional target однозначно определяется input.

При crossings или другой ambiguity одна point (x, τ) может соответствовать нескольким targets.

Тогда irreducible conditional variance остаётся даже у идеального MSE predictor.

«Conditional expectation theorem уже доказал Flow Matching»

Нет.

Он доказал regression projection.

Statement о том, что resulting field генерирует marginal path, является отдельным transport result.

«На training segment model знает, по какому segment продолжать»

Нет.

Pair identity не входит в input $v_\theta(x, \tau)$.

После marginalization эта информация специально исчезает.

Что теперь понятно

Conditional Flow Matching можно обучать на легко доступных pair-specific velocities.

Squared-error regression автоматически превращает эти hidden velocities в single-valued field, зависящий только от текущего state и time.

Flow Matching theorem говорит, что такое marginalization не ломает probability path.

Но вместе с этим мы получили важное следствие: если conditional straight segments конфликтуют друг с другом, marginal field не обязано идти вдоль каждого из них.

Именно поэтому straight interpolation ещё не означает straight learned trajectory.

4. Rectified Flow и straightness

Зачем эта тема здесь

Straight interpolation выглядит настолько простой, что легко сделать лишний вывод:

мы обучались на straight lines, значит learned flow тоже состоит из straight lines.

Предыдущий раздел уже показывает, почему это неверно.

Если несколько training segments требуют разных velocities в одном (x, τ) , model учит их conditional mean. После этого sampling trajectory определяется уже marginal field.

Rectified Flow как раз интересен тем, что делает straightness центральной целью конструкции, но достигает её не одной фразой «возьмём linear interpolation».

Интуиция

Полезно держать в голове две картинки.

Первая картинка - training segments.

Для каждой sampled pair (X_0, X_1) проводится straight segment $X_\tau = (1 - \tau)X_0 + \tau X_1$.

На нём нарисована constant arrow $X_1 - X_0$.

Segments могут пересекаться.

Вторая картинка - learned vector field.

В каждой point (x, τ) теперь должна быть ровно одна arrow $v_\theta(x, \tau)$.

Если в training picture через одну point проходили несколько segments с разными arrows, field вынужден их reconcile.

После этого из Z_0 запускается numerical ODE

$$\frac{dZ_\tau}{d\tau} = v_\theta(Z_\tau, \tau).$$

Trajectory Z_τ следует локальным arrows field. Она не хранит ID training pair и не обязана оставаться на каком-то исходном segment.

Понятия, которые вводятся

Нужны Rectified Flow, straight interpolation, trajectory, trajectory curvature и reflow.

Формальное изложение

Первый Rectified Flow fit

Пусть $(X_0, X_1) \sim \pi$.

Rectified Flow использует $X_\tau = (1 - \tau)X_0 + \tau X_1$ и objective

$$\mathcal{L}(\theta) = \mathbb{E} \left[\|v_\theta(X_\tau, \tau) - (X_1 - X_0)\|_2^2 \right].$$

В ideal function space optimum:

$$v^*(x, \tau) = \mathbb{E}[X_1 - X_0 \mid X_\tau = x, \tau].$$

То есть математически мы снова встречаем ту же regression projection.

Исходная Rectified Flow работа прямо разделяет linear interpolation X_τ и ODE trajectory Z_τ : первая может пересекаться с другими interpolations, тогда как well-defined ODE flow с unique solutions должен разрешить эти conflicts в single-valued field.

Почему ODE trajectories не могут повторить все crossing segments

Предположим, две trajectories проходят через одну и ту же point x в один и тот же момент τ .

ODE требует

$$\frac{dZ_\tau}{d\tau} = v_\tau(x).$$

Правая часть одна.

Значит, в этой space-time point нельзя одновременно иметь две разные instantaneous velocities.

При uniqueness conditions две ODE trajectories не могут прийти в один и тот же (x, τ) и затем разойтись по двум разным directions.

A training interpolations это могут, потому что каждая из них знает свои endpoints и вообще не обязана являться trajectory одного causal vector field.

Это очень важное различие.

Straight interpolation не накладывает straightness condition на marginal field

Рассмотрим learned trajectory

$$\frac{dZ_\tau}{d\tau} = v_\tau(Z_\tau).$$

Её velocity вдоль path: $\dot{Z}_\tau = v_\tau(Z_\tau)$.

Чтобы trajectory была straight с constant speed, velocity вдоль неё должна быть constant.

Значит,

$$\frac{d}{d\tau} v_\tau(Z_\tau) = 0.$$

Используем chain rule:

$$\begin{aligned} \frac{d}{d\tau} v_\tau(Z_\tau) &= \frac{\partial v_\tau}{\partial \tau}(Z_\tau) + J_x v_\tau(Z_\tau) \frac{dZ_\tau}{d\tau} \\ &= \frac{\partial v_\tau}{\partial \tau}(Z_\tau) + J_x v_\tau(Z_\tau) v_\tau(Z_\tau). \end{aligned}$$

Следовательно, exact constant-speed straightness требует вдоль trajectory

$$\boxed{\frac{\partial v_\tau}{\partial \tau} + J_x v_\tau v_\tau = 0}.$$

Straight pairwise targets сами по себе не заставляют fitted marginal field удовлетворять этому relation во всех relevant points.

Поэтому логический переход

$$X_\tau \text{ straight} \implies Z_\tau \text{ straight}$$

неверен.

Поэтому straight training interpolation и trajectory learned field нельзя отождествлять: straightness learned trajectory требует отдельного аргумента или дополнительной процедуры.

Что называем trajectory curvature

При ненулевой скорости геометрическая кривизна равна $\kappa = \|P_{v^\perp} a\| / \|v\|^2$ и от регулярной перепараметризации не зависит. Полное ускорение a дополнительно отражает изменение величины скорости, поэтому кривизной не является.

Для нашей задачи достаточно более простого diagnostic: меняется ли velocity вдоль trajectory.

Acceleration:

$$\ddot{Z}_\tau = \frac{\partial v_\tau}{\partial \tau}(Z_\tau) + J_x v_\tau(Z_\tau) v_\tau(Z_\tau).$$

Если она zero, trajectory имеет constant velocity и является straight line.

Если velocity заметно меняет direction, trajectory curved, и coarse numerical discretization обычно становится сложнее.

Отношение длины пути к хорде

В демонстрациях занятия используется простая величина

$$R = \frac{L}{\|Z_1 - Z_0\|}, \quad L = \int_0^1 \|\dot{Z}_\tau\| d\tau,$$

определённая при ненулевой хорде. Это избыточная длина пути, а не кривизна. При движении по отрезку без возвратов $R = 1$; при движении с возвратами по той же прямой $R > 1$. Постоянство скорости R не проверяет: для $Z_\tau = Z_0 + \tau^2 c$ отношение равно 1, хотя один шаг Эйлера из нуля конечной точки не даёт. Монотонной связи с геометрической кривизной у R тоже нет.

Не нужно понимать это как theorem «больше acceleration всегда означает пропорционально больше NFE». Solver error зависит от большего числа properties dynamics. Здесь важна качественная связь: perfectly straight constant-speed flow - очень простой случай для integration.

Reflow как предостерегающий пример

Straight conditional interpolation сама по себе не гарантирует straight learned trajectory. Reflow - дополнительный stage: learned flow индуцирует новый endpoint coupling, после чего model можно повторно fit на straight interpolations уже этих пар. Главное здесь - различать initial coupling, learned trajectory и coupling после reflow.

После первого training получено field $v_\theta^{(1)}$.

Теперь делаем уже настоящий sampling из этого field.

Берём $Z_0^{(1)} \sim p_0$ и интегрируем

$$\frac{dZ_\tau^{(1)}}{d\tau} = v_\theta^{(1)}(Z_\tau^{(1)}, \tau).$$

Получаем endpoint $Z_1^{(1)}$.

Пара $(Z_0^{(1)}, Z_1^{(1)})$ задаёт **новый coupling** source и target distributions.

Теперь строим уже новые straight training interpolations:

$$\tilde{X}_\tau = (1 - \tau)Z_0^{(1)} + \tau Z_1^{(1)}.$$

И снова обучаем velocity model на target $Z_1^{(1)} - Z_0^{(1)}$.

Это и есть следующий rectification / reflow stage.

Ключевая идея: endpoints теперь спарены не arbitrary initial coupling, а предыдущим deterministic flow.

Поэтому новый coupling лучше согласован с trajectories, которые вообще может реализовать ODE flow.

В идеализированной постановке Rectified Flow recursive rectification даёт increasingly straight flows; paper формулирует это как отдельное свойство reflow, а не как автоматическое следствие первого straight interpolation fit.

Что reflow меняет, а что сохраняет

Reflow **меняет coupling**.

То есть correspondence между конкретными source и target samples становится другой.

При ideal exact transport endpoint marginals при этом остаются теми же: $Z_0^{(1)} \sim p_0$, $Z_1^{(1)} \sim p_1$.

Это важная идея: generative model должна попасть в правильное **distribution**, а не воспроизвести arbitrary initial matching между отдельными source и data samples.

Геометрическая картинка словами

Вернёмся к двум crossing segments из предыдущего раздела.

Первый идёт с левого нижнего угла в правый верхний.

Второй - с левого верхнего в правый нижний.

На бумаге это буква X.

Эти training segments пересекаются в центре.

Теперь представим learned ODE field.

Через центр в фиксированный момент времени может проходить одна local arrow. Поэтому ODE не может сохранить обе диагональные instructions одновременно.

Вместо буквы X learned trajectories могут «переподключиться» так, чтобы curves не конфликтовали. Например, source point, который в исходном arbitrary coupling был связан с одним target, после marginalization может фактически прийти к другому target region.

Оговорка к самому примеру. Дискретный крест из 4 точек показывает конфликт целевых скоростей, но регулярным потоком не является: в середине обе точки сливаются в одну, и обойти это поле не может. Чтобы иллюстрировать перенос распределений, точки надо заменить распределениями ненулевой ширины. Тогда усреднённое поле способно сохранять промежуточные распределения, не сохраняя исходные пары.

Первый training picture задавал **marginals плюс arbitrary pair identities**.

Learned marginal flow сохраняет нужные marginals, но pair identities ему сохранять не требуется.

Reflow берёт новые pair identities уже из самого learned flow. Если провести straight segments между этими новыми endpoints, они обычно лучше согласованы с non-crossing dynamics. После повторного fit trajectories могут стать прямее.

Где здесь легко ошибиться

«**Rectified Flow означает straight learned trajectories**»

Это слишком сильное утверждение.

На первом training stage straight именно interpolation targets.

Learned marginal ODE trajectories могут быть curved.

«Если loss обучена идеально, trajectory обязана совпасть с training segment»

Нет, если conditional velocity неоднозначна при заданном (x, τ) .

Идеальный MSE solution тогда равен conditional mean, а не каждому individual target.

«Reflow - просто ещё несколько epochs той же training data»

Нет.

Главное изменение не число optimizer steps.

Reflow строит **новый coupling**, получая paired endpoints из предыдущего learned flow.

«Reflow и distillation - одно»

Нет.

Reflow переопределяет coupling и заново учит flow.

Distillation обычно пытается приблизить уже полученное mapping меньшим числом evaluations или отдельной direct model.

Это разные операции и разные claims.

«Straight training segments можно один раз построить, после чего one-step generation гарантирована»

Нет.

В исходной Rectified Flow работе one-step или very-low-step results связаны с properties конкретно обученного/reflowed field и experiments авторов. Это не theorem о любом model, обученном с linear interpolation.

Что теперь понятно

Straight interpolation нужна потому, что даёт максимально простой conditional target.

No simplicity conditional targets и simplicity marginal ODE - разные вещи.

Reflow является дополнительным шагом: он меняет coupling так, чтобы новые conditional straight paths могли лучше согласовываться с causal non-crossing flow.

Теперь осталось понять practical side: даже если field хорошее, как именно из него получить sample на компьютере?

5. Sampling, solver и NFE

Зачем эта тема здесь

После обучения у нас есть функция $v_\theta(x, \tau)$.

Но function сама по себе ещё не выдаёт final sample.

Нужно решить initial value problem:

$$\frac{dx_\tau}{d\tau} = v_\theta(x_\tau, \tau),$$

$$x_0 \sim p_0.$$

Для neural field аналитического solution почти никогда нет.

Поэтому sampling становится numerical integration problem.

И здесь появляется следующая типичная путаница: число solver steps и число evaluations neural network - не одно и то же.

Интуиция

Представим траекторию как неизвестную гладкую кривую.

Euler method смотрит на одну local arrow и делает маленький шаг вдоль неё.

Higher-order methods могут внутри одного большого шага посмотреть на field несколько раз: в начале interval, около середины, ближе к концу. За счёт этого они лучше оценивают bending trajectory.

Так что можно иметь:

- мало solver steps, но много model evaluations внутри каждого;
- много простых Euler steps, каждый с одним model evaluation.

Если сравнивать только слово «steps», легко получить бессмысленную таблицу.

Понятия, которые вводятся

Нужны numerical ODE solver, Euler step, solver step, NFE и low-NFE sampling.

Формальное изложение

Euler из определения derivative

Пусть сейчас $x^{(m)} \approx x(\tau_m)$.

По definition ODE

$$\frac{dx}{d\tau}(\tau_m) = v_\theta(x(\tau_m), \tau_m).$$

Для малого $\Delta\tau$ Taylor expansion первого порядка даёт

$$x(\tau_m + \Delta\tau) = x(\tau_m) + \Delta\tau \frac{dx}{d\tau}(\tau_m) + O(\Delta\tau^2).$$

Подставляем ODE:

$$x(\tau_m + \Delta\tau) = x(\tau_m) + \Delta\tau v_\theta(x(\tau_m), \tau_m) + O(\Delta\tau^2).$$

Отбрасываем higher-order term:

$$x^{(m+1)} = x^{(m)} + \Delta\tau v_\theta(x^{(m)}, \tau_m).$$

Это explicit Euler method.

Если grid uniform и содержит M intervals,

$$\Delta\tau = \frac{1}{M}.$$

Euler step и NFE

Для одного Euler step нужно вычислить $v_\theta(x^{(m)}, \tau_m)$ ровно один раз.

Поэтому для plain Euler: $N_{\text{FE}} = N_{\text{steps}}$.

Но это equality конкретного solver, а не definition NFE.

Higher-order solver

Рассмотрим classical fourth-order Runge-Kutta - RK4 - для одного step размера h .

Он строит четыре field evaluations: $k_1 = v_\theta(x^{(m)}, \tau_m)$,

$$k_2 = v_\theta\left(x^{(m)} + \frac{h}{2}k_1, \tau_m + \frac{h}{2}\right),$$

$$k_3 = v_\theta\left(x^{(m)} + \frac{h}{2}k_2, \tau_m + \frac{h}{2}\right),$$

$$k_4 = v_\theta(x^{(m)} + hk_3, \tau_m + h).$$

После этого выполняется один macro update:

$$x^{(m+1)} = x^{(m)} + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

Получается: 1 solver step = 4 NFE для этой простой fixed-step реализации RK4.

Поэтому, например, 5 RK4 steps = 20 NFE, тогда как 20 Euler steps = 20 NFE.

Число solver steps различается в четыре раза, NFE одинаковое.

Для adaptive solvers relationship ещё менее прямое: solver может менять step size, отклонять неудачный proposed step, использовать несколько internal stages и иногда переиспользовать ранее вычисленную stage.

Почему NFE полезнее слова «steps»

Основная дорогая операция в генеративной модели обычно - вычисление нейросетевого поля.

Поэтому NFE лучше отражает, сколько раз запускается backbone.

Но NFE всё равно не является полным measure cost.

Один evaluation большой multimodal Transformer и один evaluation маленького U-Net - совершенно разные F , B_{HBM} , workspace и latency.

Кроме того, batch structure, compilation и hardware utilization тоже остаются важны.

NFE - отдельная координата cost vector, а не его замена.

Численный пример: одинаковый NFE, разные solvers

Возьмём простой ODE

$$\frac{dx}{d\tau} = -x,$$

$$x(0) = 1.$$

Exact solution: $x(\tau) = e^{-\tau}$.

Поэтому $x(1) = e^{-1} \approx 0,367879$.

Четыре Euler steps

Возьмём

$$h = \frac{1}{4}.$$

Один step:

$$x^{(m+1)} = x^{(m)} - \frac{1}{4}x^{(m)} = \frac{3}{4}x^{(m)}.$$

Через четыре steps:

$$x^{(4)} = \left(\frac{3}{4}\right)^4 = \frac{81}{256} \approx 0,316406.$$

Использовано: $N_{\text{FE}} = 4$.

Один RK4 step

Теперь сделаем один RK4 step с $h = 1$.

Получаем $k_1 = -1$,

$$k_2 = -\frac{1}{2},$$

$$k_3 = -\frac{3}{4},$$

$$k_4 = -\frac{1}{4}.$$

Тогда

$$\begin{aligned} x^{(1)} &= 1 + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4) \\ &= 1 - \frac{3,75}{6} = 0,375. \end{aligned}$$

Тоже использовано $N_{FE} = 4$.

Но приближения разные: 0,316406 и 0,375.

Точный ответ: 0,367879.

На этом конкретном problem один RK4 step оказался гораздо точнее четырёх Euler steps при том же NFE.

Из одного примера не следует, что RK4 всегда лучший generative sampler. Он показывает только нужный conceptual point:

NFE и solver design являются независимыми частями sampling procedure.

Откуда берётся ошибка при sampling

Чем меряется расхождение выборок

Во втором демонстрационном ноутбуке расхождение двух выборок считается эмпирической энергетической дистанцией

$$\widehat{ED} = \frac{2}{nm} \sum_{i,j} \|x_i - y_j\| - \frac{1}{n^2} \sum_{i,i'} \|x_i - x_{i'}\| - \frac{1}{m^2} \sum_{j,j'} \|y_j - y_{j'}\|.$$

Эта оценка сравнивает выборки как распределения. Она считается по конечным выборкам, поэтому даже две независимые выборки из одного и того же распределения дают заметно положительное значение: чистой ошибкой модели она не является.

Среднее попарное отклонение численного решения от опорного решения того же уравнения - другая величина. Она показывает точность интегрирования данного поля. Вычитанием этих двух метрик ошибки не разделяются: это разные диагностики, а не слагаемые одной суммы.

Есть как минимум два разных источника error.

Первый - **model error**.

Learned field $v_\theta(x, \tau)$ не идеально совпадает с desired marginal field $v^*(x, \tau)$.

Второй - **numerical error**.

Даже если $v_\theta = v^*$, finite-step solver только приближённо интегрирует ODE.

Поэтому low-NFE sample quality зависит одновременно от:

- того, насколько хорошо выучено field;
- geometry trajectories;
- выбранного solver;
- step schedule;
- числа field evaluations;
- numerical precision.

Именно поэтому statement вида

«objective X сокращает 100 шагов до 4»

почти всегда недоопределён.

Нужно спросить:

- какие именно четыре steps;
- каким solver;
- сколько NFE;
- относительно какого baseline;
- после обычного training или после reflow / distillation;
- при каком quality target.

Где помогает straightness

Если learned trajectory действительно имеет constant velocity, $Z_\tau = Z_0 + \tau c$, то один Euler step размера 1 даёт $Z_1 = Z_0 + c$, то есть exact endpoint.

Поэтому straightening полезно для coarse integration.

Но conditional training lines being straight ещё не гарантирует, что learned marginal trajectories находятся в этом regime.

Где здесь легко ошибиться

«Четыре sampling steps означает четыре NFE»

Только для solver с одним field evaluation на step, например plain Euler.

Для multi-stage methods это неверно.

«Higher-order solver всегда дороже»

Нужно назвать coordinate.

Он может иметь больше NFE на solver step, но требовать меньше steps для той же accuracy.

Wall-clock outcome зависит от задачи и implementation.

«Если два метода используют одинаковое NFE, их inference cost одинаков»

Нет.

Один field evaluation может иметь разный FLOP count, memory traffic, workspace и batching behavior.
NFE только одна coordinate cost vector.

«Flow Matching уменьшает число sampling steps»

Не как class-level theorem.

Flow Matching задаёт training framework для vector field.

Few-step performance зависит от chosen path, coupling, learned field, straightness, numerical method и возможных additional procedures.

«Rectified Flow означает one-step generation»

Тоже нет.

Straightness является motivation и может улучшаться через reflow; конкретные one-step results относятся к конкретным trained systems и conditions. Исходная Rectified Flow работа отдельно показывает reflow и experiments с coarse discretization.

«Iterative whole-object generation почти не требует памяти»

Нет.

Current x_T действительно не является growing prefix cache. Но model evaluation может требовать значительный workspace и обрабатывать большой object целиком.

Отсутствие autoregressive KV-cache не превращается в universal constant-memory claim.

Это особенно важно сохранить перед занятиями про diffusion language models.

Что теперь понятно

Training и sampling здесь разделяются очень чисто.

Во время CFM training intermediate state и target можно построить аналитически, без ODE simulation.

После training sample получается интегрированием learned ODE.

Число solver steps описывает discretization algorithm.

NFE описывает число field evaluations.

Ни одно из них отдельно не описывает полную стоимость inference.

Главная схема занятия

В первой половине курса основная схема была такой:

prefix $\rightarrow$ state representing prefix $\rightarrow$
next-token computation $\rightarrow$ longer prefix.

Теперь схема другая:

$X_0 \sim p_0 \rightarrow$ probability path $p_\tau \rightarrow$ conditional paths
 $\rightarrow$ conditional velocities $\rightarrow$ MSE regression $\rightarrow v^*(x, \tau)$
 $\rightarrow$ numerical ODE solve $\rightarrow X_1 \sim p_1$.

Самый тонкий переход находится посередине:

conditional velocities $\rightarrow \mathbb{E}[U_\tau \mid X_\tau = x]$.

Самая частая геометрическая ошибка находится сразу после него:

straight conditional interpolation $\nRightarrow$ straight learned trajectory.

А основной cost transition занятия такой:

autoregressive decode cost $\rightarrow$
iterative whole-object sampling with explicit N_{FE} .

Вопросы для самопроверки

1. В чём принципиальная разница между S_t первых занятий и X_τ здесь?

Ответ.

S_t - representation уже обработанного autoregressive prefix.

X_τ - сам генерируемый object на промежуточном generative time.

S_t обслуживает переход к следующей token position. X_τ постепенно превращается из source sample в target sample.

2. Почему нельзя заменить τ на t и считать их одним индексом?

Ответ.

Потому что t уже закреплён за token position.

$\tau \in [0, 1]$ - continuous time generative process.

В model могут одновременно присутствовать обе coordinates.

3. Что такое probability path?

Ответ.

Family distributions $\{p_\tau\}_{\tau \in [0,1]}$ с endpoints p_0 и p_1 .

Он описывает, как меняется distribution generated object во времени.

4. Что добавляет coupling по сравнению с заданными p_0 и p_1 ?

Ответ.

Coupling задаёт joint law (X_0, X_1) и тем самым говорит, какие конкретные source и target samples считаются paired.

Одни и те же marginals допускают множество couplings.

5. Как из vector field получить trajectory?

Ответ.

Решить ODE

$$\frac{dx_\tau}{d\tau} = v_\tau(x_\tau),$$

или эквивалентно integral equation

$$x_\tau = x_0 + \int_0^\tau v_s(x_s) ds.$$

6. Почему constant vector field даёт straight trajectory?

Ответ.

Если $v_\tau(x) = c$, то $x_\tau = x_0 + \tau c$.

Velocity constant, поэтому point движется по straight line.

7. Что означает statement, что field генерирует probability path?

Ответ.

Если $X_0 \sim p_0$ и X_τ получается интегрированием этого field, то distribution X_τ должно быть p_τ для каждого τ .

8. Какой conditional target даёт interpolation $x_\tau = (1 - \tau)x_0 + \tau x_1$?

Ответ.

$$\frac{dx_\tau}{d\tau} = x_1 - x_0.$$

Target constant вдоль конкретного training segment.

9. Какой target получается для $x_\tau = a_\tau x_0 + b_\tau x_1$?

Ответ. $u_\tau = \dot{a}_\tau x_0 + \dot{b}_\tau x_1$.

Это прямое differentiation interpolation по continuous time.

10. Почему для CFM target construction не нужен ODE solver?

Ответ.

Потому что и intermediate state, и target velocity вычисляются closed-form из sampled endpoints и τ : $X_\tau = (1 - \tau)X_0 + \tau X_1$, $U_\tau = X_1 - X_0$.

ODE solver нужен при sampling learned field, а не для получения каждого training target.

11. Почему model не может просто предсказывать velocity конкретной training pair?

Ответ.

Pair не входит в input model во время sampling.

Более того, один (x, τ) может быть compatible с несколькими endpoint pairs и несколькими conditional velocities.

12. Какой predictor минимизирует MSE?

Ответ.

Для random target U и observed input Z : $f^*(Z) = \mathbb{E}[U | Z]$.

Поэтому в CFM:

$$v^*(x, \tau) = \mathbb{E}[U_\tau | X_\tau = x, \tau].$$

13. Почему cross term исчезает в MSE decomposition?

Ответ.

Потому что при $m(Z) = \mathbb{E}[U | Z]$ выполняется $\mathbb{E}[U - m(Z) | Z] = 0$.

Следовательно, conditional residual ортогонален любой function от Z в L_2 sense.

14. Что дополнительно утверждает Flow Matching theorem?

Ответ.

Не только то, что conditional expectation оптимальна для regression.

Theorem связывает posterior-weighted average conditional fields с marginal probability path: resulting marginal field генерирует нужные marginals.

Это именно generative part аргумента.

15. Что произойдёт с двумя equally likely velocities $(2, 2)$ и $(2, -2)$ при одном input?

Ответ.

MSE predictor вернёт

$$\frac{1}{2}(2, 2) + \frac{1}{2}(2, -2) = (2, 0).$$

То есть marginal field может не совпадать ни с одной individual conditional velocity.

16. Почему straight interpolation не гарантирует straight learned trajectory?

Ответ.

Interpolation знает endpoints конкретной pair и имеет velocity $X_1 - X_0$.

Learned field знает только текущие (x, τ) и возвращает conditional average velocities.

При ambiguous / crossing paths этот field отличается от individual segment directions, поэтому его integral curves могут bend.

17. Какое условие соответствует constant velocity вдоль learned trajectory?

Ответ.

Для $\dot{Z}_\tau = v_\tau(Z_\tau)$ нужно

$$\frac{d}{d\tau} v_\tau(Z_\tau) = 0.$$

По chain rule:

$$\frac{\partial v_\tau}{\partial \tau} + J_x v_\tau v_\tau = 0$$

вдоль trajectory.

18. Что меняет reflow?

Ответ.

Coupling.

Первый learned flow создаёт new paired endpoints (Z_0, Z_1) . Затем model переобучается на straight interpolations уже между этими endpoints.

19. Почему reflow не равен дополнительным epochs?

Ответ.

Потому что training distribution endpoint pairs меняется.

Это новая regression problem на новом coupling, а не просто продолжение optimization прежней loss.

20. Как выводится Euler update?

Ответ.

Из first-order expansion

$$x(\tau + \Delta\tau) = x(\tau) + \Delta\tau \dot{x}(\tau) + O(\Delta\tau^2)$$

и ODE $\dot{x}(\tau) = v_\theta(x(\tau), \tau)$.

После отбрасывания higher-order term:

$$x^{(m+1)} = x^{(m)} + \Delta\tau v_\theta(x^{(m)}, \tau_m).$$

21. Чем solver step отличается от NFE?

Ответ.

Solver step - один macro advance numerical integrator.

NFE - число вызовов vector field.

Euler обычно использует один NFE на step. Classical RK4 - четыре NFE на step.

22. Сколько NFE требуют пять fixed RK4 steps?

Ответ.

В стандартной four-stage RK4 implementation: $5 \times 4 = 20$ field evaluations.

То есть 5 solver steps, но 20 NFE.

23. Почему adaptive solver может иметь NFE, не выводимое из числа accepted steps простой константой?

Ответ.

Он может использовать несколько internal stages, менять step size и отклонять proposed steps.

Rejected attempt тоже уже потратил field evaluations.

24. От чего зависит low-NFE quality?

Ответ.

Как минимум от approximation error learned field, geometry trajectories, numerical solver, discretization schedule и precision.

Поэтому low NFE нельзя приписать одному training objective без условий.

25. Можно ли сказать, что у flow-based models мало памяти?

Ответ.

Нет как class-level statement.

Flow formulation задаёт generative dynamics, но не architecture neural field.

Field evaluation может иметь любой substantial weights memory, workspace, attention state и memory traffic.

26. Что в этом занятии означает «генерация всего объекта целиком»?

Ответ.

На каждом generative time τ существует full current object X_τ той же semantic role и dimensionality, что initial/final representation.

Следующий update уточняет этот объект, а не дописывает к нему один новый autoregressive token.

Что читать дальше

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, Matt Le. *Flow Matching for Generative Modeling*. arXiv v1 - 6 октября 2022, ICLR 2023.

arXiv:2210.02747 - Это основной текст занятия. Читать прежде всего sections про CNF preliminaries, construction marginal vector field из conditional fields и CFM objective. Самое важное - увидеть, что simulation-free относится к training objective, а не к отсутствию ODE при sampling. Работа вышла в 2022 году и была опубликована на ICLR 2023.

Xingchao Liu, Chengyue Gong, Qiang Liu. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. arXiv v1 - 7 сентября 2022, ICLR 2023.

arXiv:2209.03003 - Читать ради Figure 2 и sections про linear interpolation, marginal-preserving property, non-crossing intuition и reflow. Особенно полезно посмотреть рядом picture initial crossing interpolations и trajectories rectified flow - она почти полностью закрывает типичную ошибку «straight training target означает straight trajectory».

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, David Duvenaud. *Neural Ordinary Differential Equations*. NeurIPS 2018.

arXiv:1806.07366 - Это background, если хочется нормально разобраться, откуда берутся CNF и instantaneous change of variables. Для текущего занятия достаточно ODE и flow sections; adjoint training можно пока пропустить.

Yaron Lipman et al. *Flow Matching Guide and Code*. 2024.

arXiv:2412.06264 - Это уже не foundational paper, а self-contained guide. Полезен после занятия, если исходная paper кажется слишком плотной: там единым языком собраны probability paths, vector fields, couplings и practical design choices.

Пока не нужно заранее уходить глубоко в score matching и SDE. На следующем занятии мы специально введём их с нуля и только потом аккуратно сопоставим diffusion probability paths с уже известным здесь transport language.

Переход к занятию 5

Теперь есть deterministic transport picture:

$$\begin{aligned} X_0 \sim p_0 &\longrightarrow \frac{dX_\tau}{d\tau} = v_\tau(X_\tau) \\ &\longrightarrow X_1 \sim p_1. \end{aligned}$$

Мы научились задавать simple conditional paths, получать из них regression targets и понимать, почему conditional regression восстанавливает marginal field.

Но пока path выбирался напрямую.

Следующий вопрос:

что произойдёт, если строить промежуточные distributions через controlled Gaussian corruption и описывать их не только velocity field, но и gradient log-density?

Отсюда появятся diffusion, score $\nabla_x \log p_\tau(x)$,

SDE и probability-flow ODE.

Важно, что следующий материал не будет объявлен «тем же самым другими словами». Связь между diffusion и flow matching требует условий на probability path, parameterization и weighting objective.

Что этот конспект добавляет к контракту нотации

Все соглашения занятий 1-3 сохраняются.

В частности, t остаётся индексом token position. Он не получает нового continuous meaning.

Новое continuous time генеративного процесса обозначается

$$\tau \in [0, 1].$$

Это продолжает notation transition, зафиксированный в конце занятия 3.

Source и target distributions: $p_0, \quad p_1$.

Probability path: $p_\tau, \quad \tau \in [0, 1]$.

Random variables пишутся с uppercase: $X_0, \quad X_\tau, \quad X_1$.

Конкретные realizations - с lowercase: $x_0, \quad x_\tau, \quad x_1$.

Coupling endpoints обозначается $(X_0, X_1) \sim \pi$.

Learned marginal vector field: $v_\theta(x, \tau)$.

Conditional training velocity: u_τ или, когда conditioning нужно показать явно, $u_\tau(x \mid c)$.

Для straight endpoint interpolation: $X_\tau = (1 - \tau)X_0 + \tau X_1$, $U_\tau = X_1 - X_0$.

Training interpolation и learned ODE trajectory дальше различаются notation: X_τ

- random interpolation, используемая для construction training examples; Z_τ
- trajectory, полученная integration learned field.

Это distinction нельзя молча стирать.

Для general interpolation этого занятия используются a_τ, b_τ : $X_\tau = a_\tau X_0 + b_\tau X_1$.

Они являются functions **continuous time** τ и не переопределяют α_t, β_t , введённые раньше для token-indexed memory updates.

Index numerical solver step обозначается m : $x^{(m)}$.

Step size: $\Delta\tau$ или локально h .

N_{FE} не является новым symbol: эта coordinate уже присутствовала в cost vector занятия 1. Начиная с этого занятия она становится operational quantity и означает число evaluations learned field при generation одного result. Сам cost vector не меняется.

Главное новое содержательное соглашение такое:

conditional path, marginal probability path, training interpolation, learned vector field и numerical trajectory - разные уровни описания. Совпадение или простота одного уровня не разрешает переносить свойство на остальные без отдельного аргумента.